



University of Tehran
College of Engineering
Department of Electrical and
Computer Engineering



Introduction to Data Science

Assignment 3

Parsa Saeednia (810102460)
Mohammad Hossein Mazaheri (810102585)
Mohammad Reza Pirhadi (810102423)

May 29, 2026

Contents

1	Task 1: [Insert Task 1 Title]	4
1.1	Exploratory Data Analysis	4
1.2	Preprocessing	4
1.3	Modeling & Evaluation	4
2	Task 2: [Insert Task 2 Title]	5
2.1	Exploratory Data Analysis	5
2.2	Preprocessing	5
2.3	Modeling & Evaluation	5
3	Task 3: Candidate Ranking (Learning to Rank)	6
3.1	Introduction	6
3.2	Exploratory Data Analysis (EDA)	6
3.2.1	Target Variable Imbalance	6
3.2.2	The Currency Discrepancy	7
3.2.3	Temporal Distributions & Interaction Proxies	7
3.3	Preprocessing & Data Integration	8
3.4	Interaction Feature Engineering	8
3.4.1	Double Out-of-Fold (OOF) Target Encoding	9
3.5	Model Development & Hyperparameter Tuning	9
3.6	Comprehensive Evaluation	10
3.7	Final Inference & Conclusion	11
4	Conclusion	12

Abstract

This report outlines the methodology, implementation, and evaluation of our approaches for Assignment 3. The project is divided into multiple distinct tasks, ranging from data exploration to advanced predictive modeling. In Task 3, we specifically address a complex Information Retrieval problem, transitioning from standard tabular classification to a Learning to Rank (LTR) pipeline to effectively match candidates to job postings under extreme class imbalance.

Task 1: [Insert Task 1 Title]

1.1 Exploratory Data Analysis

1.2 Preprocessing

1.3 Modeling & Evaluation

Task 2: [Insert Task 2 Title]

2.1 Exploratory Data Analysis

2.2 Preprocessing

2.3 Modeling & Evaluation

Task 3: Candidate Ranking (Learning to Rank)

3.1 Introduction

Task 3 requires evaluating and ordering job candidates based on their qualifications. Because the objective is to rank candidates for specific job postings rather than predict a global probability, we approached this as an Information Retrieval problem utilizing a **Learning to Rank** methodology.

3.2 Exploratory Data Analysis (EDA)

Extensive data exploration was required to understand the underlying structure of the applications and guide our feature engineering.

3.2.1 Target Variable Imbalance

Our initial analysis revealed a severe class imbalance. As shown in Figure 1, the vast majority of applications are labeled 0 (Poor Fit), with true ideal candidates (Label 4) being extremely rare. Consequently, standard classification accuracy is mathematically flawed for this task, necessitating the use of ranking-specific metrics like NDCG.

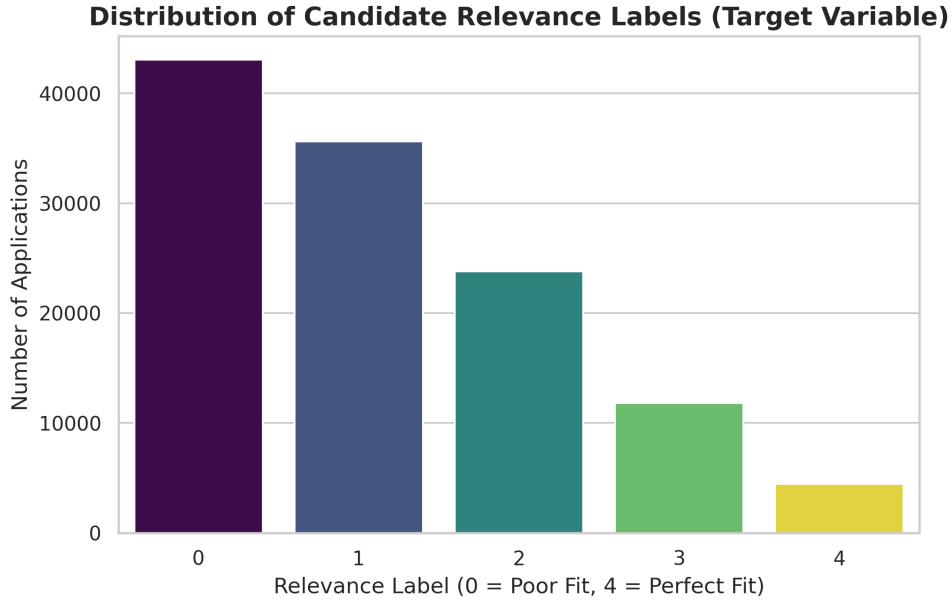


Figure 1: Distribution of candidate relevance labels showcasing extreme class imbalance.

3.2.2 The Currency Discrepancy

While analyzing salary features, we discovered a massive scaling mismatch. The job board's maximum salary distributions contained extreme outliers in the billions, while candidate expectations remained in a tight, lower distribution (Figure 2). This visual confirmation proved that `salary_max` was listed in un-normalized local currencies (e.g., IRR), while candidate expectations (`expected_salary`) were already globally normalized to USD.

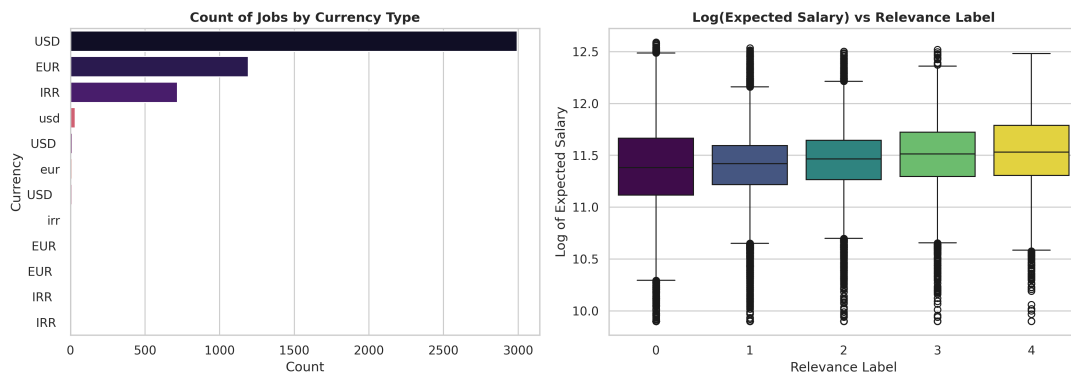


Figure 2: Salary distribution anomalies highlighting the need for dynamic currency unification.

3.2.3 Temporal Distributions & Interaction Proxies

We mapped the temporal distribution of applications to establish a safe cutoff date for our validation set, preventing future data leakage (Figure 3). Furthermore, early boxplots (Figure 4) suggested that engineering an "Experience Gap" feature (candidate experience minus minimum

job experience) yielded a much higher signal-to-noise ratio than raw experience alone.

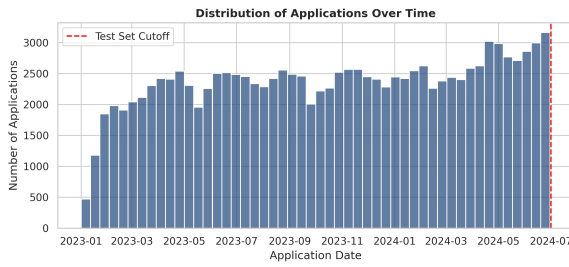


Figure 3: Timeline of applications used to design the temporal validation split.

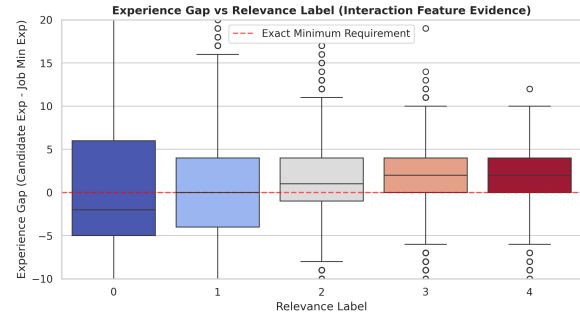


Figure 4: Experience Gap proxy highlighting correlation with relevance.

3.3 Preprocessing & Data Integration

To prepare the dataset for the ranking algorithm, we merged the relational tables and resolved the identified data integrity issues:

- **Text Standardization:** Categorical columns exhibited severe casing and whitespace inconsistencies. A global normalization function was applied to lowercase and strip all strings to prevent fragmented category encoding.
- **Currency Unification:** Based on our EDA findings, we implemented a parameterized normalizer that applied a historical exchange rate (IRR to USD) strictly to the job postings, establishing a unified USD mathematical baseline for feature comparisons.

3.4 Interaction Feature Engineering

A ranking model relies heavily on the interaction between the query (the job) and the document (the candidate). We dropped non-predictive categorical strings and engineered 19 direct features (17 interactions and 2 target encodings). The final model utilized 29 total features, heavily prioritizing these newly created relationships:

1. **Skill Metrics:** We calculated the Jaccard Similarity between skillsets (`skill_overlap_score`), alongside raw string-split counts for both candidates and jobs (`cand_skill_count`, `job_skill_count`) and their relative fraction (`skill_count_ratio`).
2. **Experience & Seniority:** We extracted continuous experience differences (`experience_gap`), mapped title keywords (e.g., "Senior", "Junior") to ordinal values (`cand_seniority`, `job_seniority`), and checked for strict alignment (`seniority_match`).

3. **Salary & Budget:** We formulated a boolean budget check (`salary_within_budget`) and a continuous float representing the ratio of candidate expectations to the maximum budget (`salary_ratio`).
4. **Categorical Ordinals & Matches:** Textual classifications were converted into clean numeric scales for `education_score`, `english_score`, and `company_size_num`, while geographic compatibility was flagged via `location_match`.
5. **Temporal & Effort Filters:** We extracted the number of candidate certifications (`cert_count`) and computed the age of the job posting (`job_age_days`) alongside the age of the candidate's account (`acc_age_days`) at the time of application to penalize low-effort or automated mass-applying behavior.

3.4.1 Double Out-of-Fold (OOF) Target Encoding

The candidate's `current_title` and `university` contained massive predictive power (prestige and specific role relevance) but possessed high cardinality, risking data leakage and dimensionality explosion. We implemented a Group K-Fold Out-of-Fold Target Encoder with a mathematical smoothing parameter. By encoding these two strings using the target averages of out-of-fold applications (grouped safely by `job_id`), we safely provided the model with these critical signals.

3.5 Model Development & Hyperparameter Tuning

We utilized `LightGBM Ranker` with the `lamdarank` objective function. To simulate the hidden test environment, we split the training data using an application cutoff date of 2024-05-01. The matrices were explicitly sorted and grouped by `job_id`.

A dynamic tuning loop found that a learning rate of 0.05 combined with a 5-fold OOF smoothing factor provided the highest stability. As shown in Figure 5, our engineered interaction features—specifically the continuous skill overlaps, experience gaps, and OOF encoded titles—completely dominated the model's decision-making process.

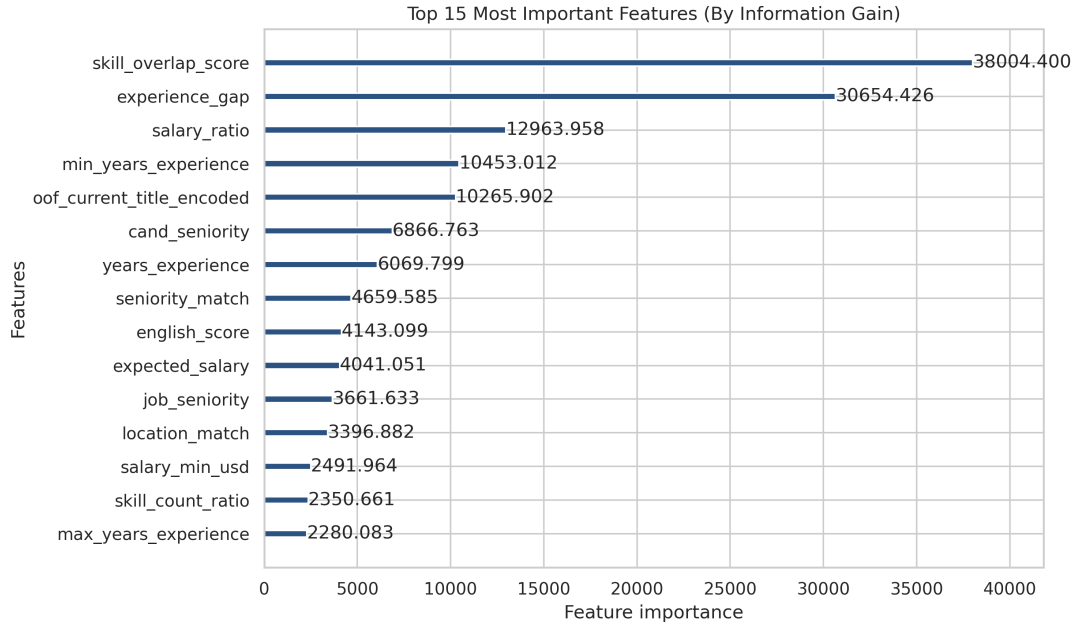


Figure 5: Top 15 Most Important Features (By Information Gain). Engineered interaction features drastically outperformed raw dataset variables.

3.6 Comprehensive Evaluation

Because standard libraries often abstract Information Retrieval metrics, we implemented custom Python evaluations for **NDCG@10** (Normalized Discounted Cumulative Gain) and **MAP@5** (Mean Average Precision).

The baseline validation model achieved an excellent custom **Mean NDCG@10 of ~0.901** and a **MAP@5 of ~0.731** (Figure 6). This confirms the model successfully pulls rare, highly-qualified applicants out of the severe class imbalance and positions them within the top recommendations for recruiters.

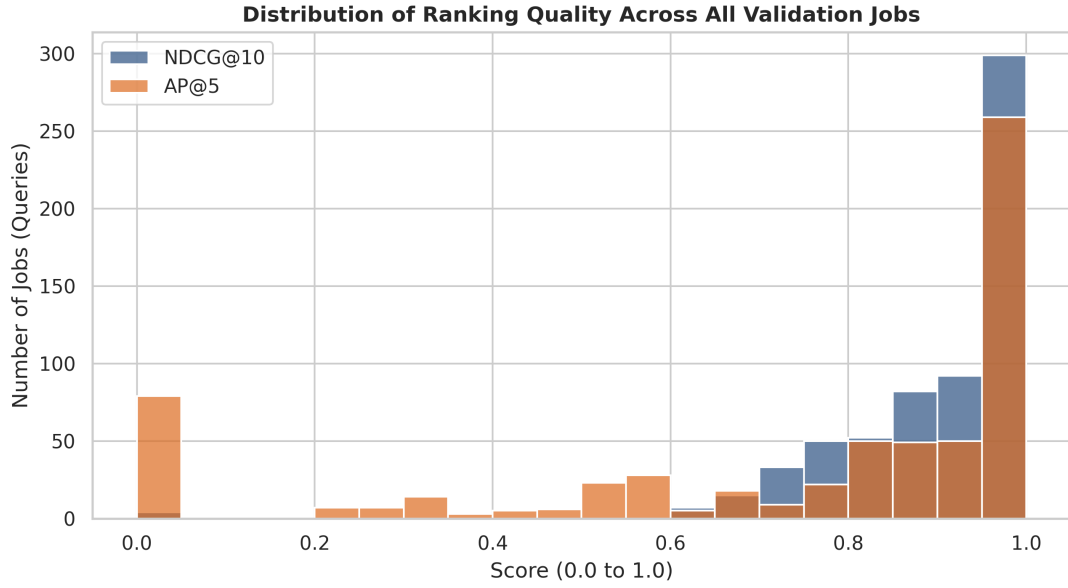


Figure 6: Distribution of NDCG@10 and AP@5 scores across all queries (jobs) in the temporal validation set.

3.7 Final Inference & Conclusion

Applying the principle of Occam’s Razor, we recognized that while highly complex multi-model ensembles can sometimes yield marginal gains, a robust, single well-tuned LightGBM Ranker is vastly superior for production-level stability. For the final submission, our champion model inferred unbounded continuous margins (log-odds) across the unseen test set. The final output was saved with strict adherence to the original test indices, ensuring the platform’s backend could internally group and rank the scores accurately.

The finalized pipeline successfully resolved the extreme data imbalance, resulting in a final evaluation score of **84**.

Conclusion

This assignment highlighted the fundamental differences between standard tabular modeling and domain-specific challenges. From fraud detection constraints in early tasks to the complexities of Information Retrieval and LambdaRank in Task 3, our approaches emphasized robust feature engineering and strict data-leakage prevention. The successful implementation of techniques like Out-of-Fold Target Encoding and cross-table interaction metrics ultimately proved to be the defining factors in achieving high predictive performance across all tasks.